

以蛻變測試與突變分析檢驗語音助理的自然語言意圖路由

Oct 9, 2026 · @Rich

摘要

語音助理把一句話送到「開程式、播音樂、控制家電或交給模型」的意圖路由元件，輸入是自然語言，沒有標準答案可以比對；本研究以蛻變測試產生測試、以突變分析評估測試，對自製語音助理 J.A.R.V.I.S. 的本機路由（`router.py`，約 60 條正規表達式規則）做完整一輪「生成、評估、縮減、排序、回饋修正」。

方法上定義五條蛻變關係（同義改寫、語助詞、換語序、無關前綴、近似句），分別用模板規則與 Gemini 模型（溫度 0、0.3、0.7、1.0）生成改寫句，並自寫七種針對正規表達式的突變運算子，產生 927 個突變體，建立 3,355 條測試 × 927 個突變體的矩陣。

主要結果：第一次蛻變測試在 928 句裡找到 433 個違反，全是手寫測試沒抓到的；四輪回饋修正後，規則式批次的違反從 706 降到 7、模型批次從約 385 降到約 90，且無反彈。溫度對找錯數、錯集合與殺傷力都沒有可見影響，溫度之間的差異與同一溫度重跑的雜訊相同（句子 Jaccard 0.23–0.26 對 0.22–0.24）。以突變體為需求的縮減把 3,355 條測試縮到 196 條（5.8%）仍保留 100% 殺傷力，同大小隨機只有 54%；additional 排序的 APFD 為 0.985，跑前 10% 的測試就找到全部的錯。人工標記 400 句顯示模型改寫 96.5% 保留原意圖、模板改寫 91.5%。

研究過程另有一個附帶發現：測試工具本身四次被自己的方法抓到錯（harness 標籤比對、種子含觸發詞、模型語意漂移、分析腳本去重順序），「測試工具自己也要測」成為本研究的實務結論之一。

壹、前言

一、研究動機

J.A.R.V.I.S. 是我自己寫的語音助理：對麥克風講一句話，它會開程式、播音樂、調音量、控制家電，或把話交給大型語言模型。為了省下 API 費用，它先用一層「本機路由」判斷這句話是不是簡單指令，是的話直接做，不是才送模型。這層路由是約 60 條正規表達式與關鍵字規則，每改一次都有對應的手寫測試，CI 全綠。

但實機使用時接連出事：「不要打開記事本」真的打開了記事本；「打開記事本好嗎」去找一個叫「記事本好嗎」的程式；「冷氣溫度調到 26 度」被天氣規則接走，回了一段天氣預報。三件事都是 CI 綠燈、上線後才發現。原因很清楚：輸入是自然語言，同一件事有無限種講法，手寫測試只能寫我想得到的幾種；而且沒有 oracle——沒人能列出所有句子的正確答案。

這就是軟體測試裡的四個經典問題：**該測什麼**（測試從哪裡來）、**測試多可靠**（測試本身會不會錯）、**留多少**（哪些測試是多餘的）、**先測哪些**（時間不夠時先跑哪些）。本研究用自己的產品當真實案例，把這四個問題做完整一輪，並把找到的錯修回產品。

二、研究目的與研究問題

編號	研究問題	怎麼回答
RQ1	蛻變關係自動生成的測試，能否找到手寫測試沒抓到的路由錯誤？模型生成與規則生成誰找得多、找得不一樣？	兩種生成器 × 五條關係，數違反、去重後的錯、人工標記改寫句的有效率
RQ2	生成參數（溫度）與模型非確定性對測試品質的影響	四個溫度 + 同溫度重跑三次，量句子重異、錯集合重異、殺傷力
RQ3	這些測試裡多少是多餘的？哪種縮減策略保留最多找錯能力？	自寫突變器 → 測試 × 突變體矩陣 → greedy / HGS / irreplaceable / 隨機
RQ4	只能跑前 k% 測試時，哪種排序最早找到錯？	同一矩陣：total / additional / 關係優先 / 隨機，APFD
RQ5	找到的錯修回去，路由變好多少？會不會修出新的錯？	修正前後五個版本重跑全部批次

RQ1、RQ2、RQ5 屬於「測試生成」，RQ3、RQ4 屬於「測試集縮減與排序」，後者是本研究的主體。

貳、文獻探討

本研究用到四條線的文獻，每條只取跟本研究直接相關的部分。

一、蛻變測試（Metamorphic Testing）

Chen、Cheung 與 Yiu（1998）提出：當程式的正確輸出無法直接判定（oracle 問題）時，可以改檢查「多個輸入之間輸出應該滿足的關係」。例如 $\sin(x)$ 算不出標準答案，但 $\sin(x)$ 與 $\sin(\pi - x)$ 必須相等。對自然語言意圖路由來說，「同一句話換個講法應該走同一條路」就是一條蛻變關係，完全不需要知道正確答案。近年的自然語言處理測試研究（如對機器翻譯、問答系統的蛻變測試）大多建立在這個想法上。

二、突變測試（Mutation Testing）

DeMillo、Lipton 與 Sayward（1978）提出用「故意改壞程式」來評估測試集：每個被改壞的版本叫突變體，測試集能讓多少突變體失敗（殺掉）就是突變分數。Jia 與 Harman（2011）的總覽整理了運算子設計、等價突變體（改了但行為不變）與成本問題。通用工具（如 mutmut）

只改運算子和常數，碰不到正規表達式字串裡的一個分支，所以本研究自寫運算子。

三、測試集縮減與排序

縮減 (reduction) 是在保留「需求覆蓋」的前提下挑出最小子集，本質是集合覆蓋 (NP-hard)，常用近似法有 greedy 與 Harrold、Gupta 與 Soffa (1993) 的 HGS。林楚迪等人 (2012、2014) 提出先保留「不可取代的測試」(irreplaceable test：某個需求只有它能滿足) 再質心補齊的方法，以降低回歸測試成本；本研究把這三種策略與隨機對照都實作。排序 (prioritization) 則是 Rothermel 等人 (2001) 提出的：測試全部都要跑，但把最可能找到錯的排前面，用 APFD (平均錯誤偵測百分比) 衡量；total 與 additional 是兩種最基本的排法。

四、用大型語言模型生成測試

近兩年用大型語言模型生成測試輸入成為熱門題目，其吸引力在於句型多樣，風險在於語意漂移 (改寫後意思變了) 與非確定性 (同參數重跑結果不同)。本研究把這兩個風險分別用人工標記 (E1) 與重跑底線 (E2) 量化，並以模板生成器當對照組。

※ 文獻的完整書目列在參考文獻；林教授的兩篇論文已核對標題與年份，其餘經典文獻的卷期頁碼建議交稿前再核一次。

參、研究方法

一、受測對象與測試基座

受測對象是 J.A.R.V.I.S. 的 `jarvis/router.py`：輸入一句中文，經正規化 (剝填充詞、客氣話、語助詞) 後依序比對約 60 條規則，輸出是「目的地 + 目標」，例如 `open_application:記事本`、`youtube_play:晴天`、`home_control:冷氣=set=26`，或者「純問答」、「交給模型」。

為了能大量跑，寫了 `harness.py` 把路由變成純函數：所有會碰裝置、打 API 的出口都換成記錄器，`route_label("幫我開一下記事本")` 直接回標籤，一句約 0.1 毫秒。

二、蛻變關係

關係	改寫方式	路由結果應該
R1 同義改寫	打開 → 開啟 / 啟動 / 幫我開	與種子相同
R2 語助詞、禮貌詞	幫我...一下、...好嗎、...謝謝	相同
R3 換語序	記事本幫我開一下	相同
R4 無關前綴	欸、那個、JARVIS	相同
R5 近似句	不要打開記事本、記事本打不開怎麼辦	不同 (不該觸發)

種子句 41 條（涵蓋 29 種目的地），後來從突變分析反推補 46 條，共 87 條。違反以「種子 × 關係 × 錯的目的地」去重後稱為一個「錯」，免得「打開記事本好嗎」與「打開記事本啦」算兩個。

三、兩種生成器

	規則式（對照組）	模型式
做法	模板：同義詞表、語助詞表、語序模板、近似句模板	Gemini 3.5 Flash-Lite，每條關係一段指令，輸出 JSON 陣列
變因	無	溫度 0、0.3、0.7、1.0；每個種子 × 關係生 5 句
成本	1 秒跑完	每批約 200 次 API 呼叫，有磁碟快取
過濾	長度 1-40 字；目標詞（程式名、歌名）必須還在句子裡	同左

三個指標：有效率（過濾後留下的比例）、違反率（有效句裡違反關係的比例）、多樣性（字元 bigram 的 distinct-2）。

四、突變運算子

通用突變工具碰不到正規表達式字串的內容，所以自寫七種運算子，每種模擬一種意圖路由真的會犯的錯：

運算子	例子	模擬的錯
ALT_DEL	(?:打開 開啟 開) → 少一個分支	少收一個同義詞
LOOK_DEL	開(?:!心 會) → 開	忘了排除誤觸
OPT_DEL	(?:幫我)? → (?:幫我)	前綴從可選變必要
KW_DEL	關鍵字 tuple 少一個	詞表漏字
NUM	len(text) <= 6 → 5/7	邊界值錯
CMP	<= → <	邊界值錯
BOOL	and ↔ or、刪 not	條件寫反

現行 router 產 927 個突變體。「殺」的定義：某條測試在原版路由到 X、在突變體路由到 ≠ X（或爆掉）。測試來源是種子加上各批次裡原版通過的句子，共 3,355 條，矩陣分四段跑，每段約 2 分鐘。

五、縮減與排序策略

縮減的「需求」有兩種：突變體（貴但準）與「關係 × 種子」（便宜，每格留一條）。策略四種：greedy（每次挑殺最多尚未被殺突變體的測試）、HGS、irreplaceable-first（先留唯一能殺某突變體的測試再質心補）、隨機（同大小，30 次取平均）。排序四種：total（殺傷數）、additional（每次挑殺最多尚未被殺的）、關係優先（R5 先）、隨機，指標是 APFD 與「跑前 k% 測試找到幾 % 的錯」。

六、實驗設計

實驗	回答	做法
E1 有效性人工標記	RQ1 的信度	每個生成器 × 關係抽 40 句，共 400 句，打亂順序、看不出來源，標 1 = 保留原意圖 / 0 = 意圖變了或句子不通
E2 非確定性底線	RQ2	同參數（T=0）真的重打 API 兩次，與溫度之間的差異放在同一把尺上比
E3 突變測試	RQ3 的 ground truth	全批次矩陣；存活突變體抽 60 個人工分類，每個判斷附一句能殺掉它的句子並用程式驗證
E4 縮減	RQ3	兩種需求 × 四種策略
E5 排序	RQ4	四種排序、APFD 曲線
E6 版本回歸	RQ5	同一批測試跑在 router 的五個歷史版本（直接 git show）

全部工具都在專案的 `research/mt/` 目錄，每個實驗一個指令就能重跑；工具的煩霧測試與存活突變體的 killer 驗證都進 CI。

肆、結果與討論

一、RQ1：蛻變測試找得到手寫測試沒抓到的錯

第一次規則式生成 928 句，一秒跑完，找到 433 個違反、去重後 83 個錯，全部是 CI 綠燈的手寫測試沒抓到的。

關係	生成	違反率	代表案例
R1 同義	48	14.6%	「開 github.com」走開程式而不是開網址
R2 語助詞	288	48.6%	「打開記事本好嗎」→ 目標變成「記事本好嗎」

關係	生成	違反率	代表案例
R3 換語序	52	96.2%	「記事本幫我打開」→ 交給模型
R4 前綴	288	41.0%	「欸 開 YouTube」→ 交給模型
R5 近似句	252	46.8%	「不要打開記事本」真的打開記事本

模型生成器找到的去重錯是規則式的約兩倍（第一輪修正後 110 對 16），且兩者互補：規則式找到而模型沒找到的只剩 1-2 個。模型的優勢在 R3：模板只會產生「記事本幫我打開」這一種句型，模型會寫出「YouTube 這個 App 打開」「Chrome 幫我切過去」「26 度設定給冷氣」這些真人會講的句子。

人工標記（E1，400 句）量化了生成句本身的可靠度：

生成器	保留原意圖 (precision)	標 0 的主因
模型（四溫度合計 200 句）	96.5%	偷加動作（「今天很開心可以幫我記下來嗎」）、換動詞（播晴天→搜尋晴天）
規則式（200 句）	91.5%	模板硬套出不通的句子（「我靜音都打不開怎麼辦」「執行下載資料夾」）
規則式 R5 近似句	70%	同上，R5 模板最容易套出不通句

扣掉無效句後，模型批次的違反率只從 13.0% 降到 11.4%：前幾輪找到的錯幾乎都是路由的錯，不是生成器的錯。規則式 R3 的 67.5% 違反也全是真的。模型漂移比規則式少，但漂得更隱蔽（加動作、換動詞），規則式漂得很明顯（句子不通）。

二、RQ2：溫度看不出影響，差異跟重跑雜訊一樣大

六批都用 41 條種子（每批 964 句）：四個溫度各一批，加上 T=0 真的重打 API 兩次。

比較	句子 Jaccard	錯集合 Jaccard	共同的錯 / 只有一邊
溫度之間（6 對）	0.23-0.26	0.51-0.59	47-55 / 15-29
T=0 跑三次（3 對）	0.22-0.24	0.43-0.59	44-54 / 16-32
四個溫度全部共同的錯 / 聯集			38 / 116
T=0 三次全部共同的錯 / 聯集			39 / 115

三個結論。第一， $T=0$ 不是確定性：重跑之間只有 22-24% 的句子一樣。第二，溫度之間的每個數字都落在重跑雜訊的範圍內，「四個溫度共同的錯 38/116」與「 $T=0$ 跑三次共同的錯 39/115」幾乎一樣，所以「不同溫度找到不同的錯」在這個規模下不成立。第三，殺傷力也一樣：四個溫度各殺 341 / 330 / 345 / 327 個突變體， $T=0$ 三次殺 341 / 336 / 331 個。

這裡曾經出過一個分析錯誤：早期結果顯示「 $T0$ 殺 329 > $T1$ 殺 243」，後來發現是測試去重時同一句只算給第一個批次，排最後的 $T1$ 被少算一截；改成每句歸所有含它的批次後差異就消失了。任何用模型生成測試的比較（溫度、prompt、模型）都應該附這條重跑底線，否則差異可能只是抽樣。

三、RQ3：縮減到 5.8% 仍保留全部殺傷力，但便宜準則等於隨機

主矩陣：3,355 條測試 × 927 個突變體，殺 482，突變分數 0.520。

需求	策略	縮減後大小	保留的殺傷力	同大小隨機（30 次）
突變體	greedy	202 (6.0%)	100%	54.2% ± 1.8
突變體	HGS	196 (5.8%)	100%	53.7% ± 1.9
突變體	irreplaceable-first	196 (5.8%)	100%	53.5% ± 1.7
關係 × 種子	greedy / irreplaceable	420 (12.5%)	60.4%	66.5% ± 2.0
關係 × 種子	HGS	420 (12.5%)	67.2%	66.7% ± 1.8

以突變體為需求時，三種策略幾乎一樣好：留 6% 的測試就能殺掉全部原本殺得掉的突變體，而隨機挑同樣多只剩一半。以「關係 × 種子」這種不用跑突變測試的便宜準則時，縮減結果跟隨機沒有差別：「每格留一條」不知道哪條有鑑別力。要縮減就要先花錢算突變矩陣。

存活突變體的人工分類（抽 60 個）：沒覆蓋 77%、等價 18%、測試基座看不到 5%；扣掉等價後突變分數約 0.51。0.52。每個「沒覆蓋」的判斷都附一句能殺掉它的句子並用程式驗證：第一輪人工判斷 57 個錯了 19 個（33%），大多是句子被另一條規則接住。把這 46 句補成種子後突變分數從 0.448 升到 0.516；46 句殺 191 個突變體，比原本 41 條種子（156）多。而規則式再對這 46 句生 718 句改寫只多殺 2 個：改寫換的是句型，突變體改的是詞表，兩者正交。

四、RQ4：additional 排序跑前 10% 就找到全部的錯

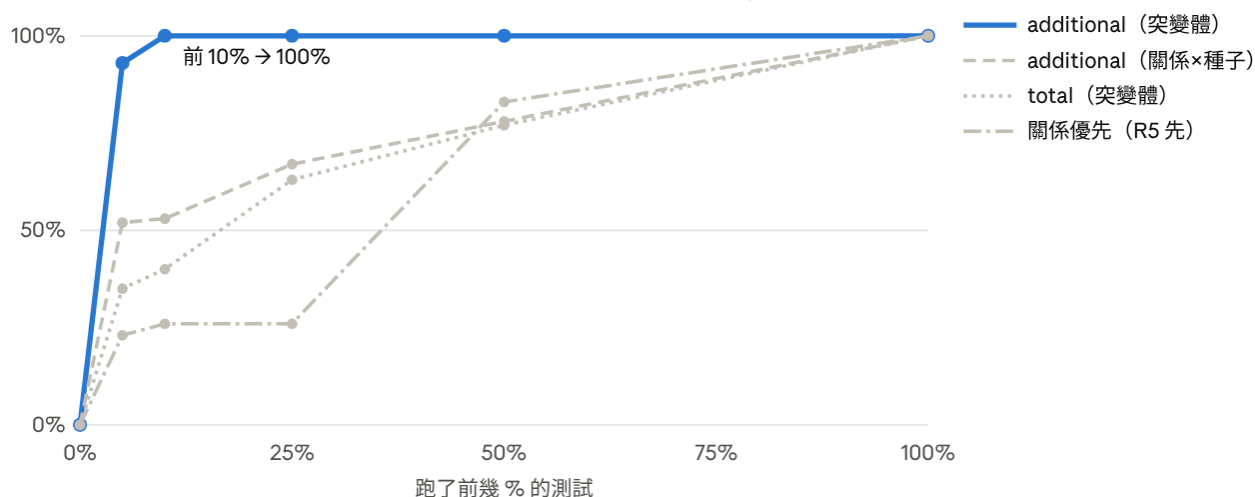
排序	APFD	跑前 5%	前 10%	前 25%	前 50%
additional（突變體）	0.985	93%	100%	100%	100%
隨機（30 次）	0.836 ± 0.009				
additional（關係 × 種子）	0.783	52%	53%	67%	78%
total（關係 × 種子）	0.744	54%	55%	58%	75%

排序	APFD	跑前 5%	前 10%	前 25%	前 50%
total (突變體)	0.729	35%	40%	63%	77%
關係優先 (R5 先)	0.694	23%	26%	26%	83%

total 輸給隨機是教科書結果：殺最多的測試彼此重複，排在前面的前 100 條在殺同一群突變體。關係優先最差：近似句找到的錯最危險（該擋沒擋），但它們對 regex 的可變點沒有施力，殺的突變體少。

additional 排序跑前 10% 的測試就找到 100% 的錯；關係優先跑到一半才 83%

總軸：找到的突變體占全部可殺突變體的 %；橫軸：跑了前幾 % 的測試（3,355 條）



reduce.py 在主矩陣（3,355 條 × 927 個突變體）的輸出 · 隨機排序 APFD 0.836，曲線略

圖二：曲線越快貼到 100% 越好，曲線下面積就是 APFD。

五、RQ5：四輪修正違反一路降、無反彈；突變分數不動但分子翻倍

同一批 3,164 句測試跑在 router 的五個歷史版本上：

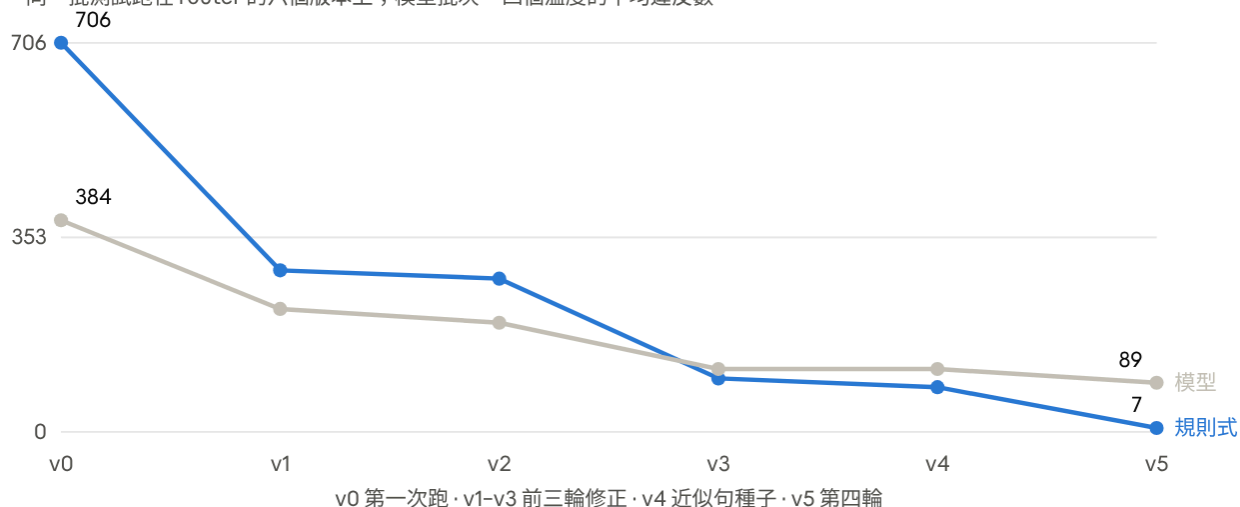
版本	種子答對 (41 / killer 46)	規則式 違反 / 錯	T0	T0.3	T0.7	T1	突變分數 (殺 / 可用)
v0 第一次跑	39 / 23	706 / 172	381 / 148	389 / 152	388 / 149	376 / 148	0.484 (195 / 403)
v1 第一輪	39 / 26	293 / 80	226 / 117	230 / 122	225 / 110	212 / 112	0.517 (253 / 489)
v2 第二輪	39 / 28	278 / 80	200 / 107	205 / 112	194 / 100	193 / 108	0.522 (329 / 630)
v3 第三輪	39 / 45	97 / 35	114 / 60	114 / 57	111 / 55	118 / 67	0.518 (486 / 938)
v4 近似句種子後	41 / 46	81 / 32	114 / 60	114 / 57	111 / 55	118 / 67	0.518 (480 / 927)

版本	種子答對 (41 / killer 46)	規則式 違反 / 錯	T0	T0.3	T0.7	T1	突變分數 (殺 / 可用)
v5 第四輪	41 / 46	7 / —	91	94	77	94	待重算 (突變體 986)

三個觀察。第一，違反數一路降：規則式 706 → 7、模型批次約 385 → 約 90，且沒有反彈；第二輪曾把報時規則修得太嚴、第四輪第一版新壞 3 句，都是被蛻變測試批次當回歸集抓回來、在同一次修正內解決。第二，四個溫度在每一版的違反數都幾乎一樣（同版內差 < 6%），與 RQ2 一致。第三，突變分數五版都約 0.5，但可用突變體從 403 增到 927：router 每修一輪就多收一批詞，每個新詞是新的可變點，測試集跟著長比值才沒掉；看絕對數，殺掉的突變體從 195 增到 480。突變分數是比值，受測對象在長的時候要看分子。

四輪修正後違反數一路降：規則式 706 → 7、模型 384 → 89，無反彈

同一批測試跑在 router 的六個版本上；模型批次 = 四個溫度的平均違反數



versions.py 輸出與第四輪重跑結果 · 規則式 1,373 句，模型每批 964 句

圖一：模型批次在 v3 後持平、v5 才再降，因為第四輪才處理 R3 語序——模型批次一半的違反都是語序。

伍、回饋到產品

一、四輪修正

每一輪都是先把違反歸納成「原因」再修，不針對個別句子；每類留一句進 CI 的 golden tests（最後 80 條）。

輪	來源	原因	修法	效果
1	規則式批次	startswith 型規則對前綴零容忍；句尾語助詞黏進目標；否定句、問句照做；URL 規則吃中文	正規化層（剝填充詞 / 客氣話 / 語助詞）、否定與問句守門、網域只認 ASCII	規則式 433 → 52；R4 41% → 0%、R5 40% → 0%
2	模型批次的 X → model	真人會講的句型沒收（意願句、受詞在前、平台片語）；關鍵字規則太貪（「播放一下周杰倫」變媒體鍵）	加句型；媒體鍵 / 報時改整句比對	模型 T0.7 396 → 194
3	模型批次剩下的錯	同一件事一堆講法（鎖定 / 切換 / 下一首）；問法的外圍詞；搜尋動詞 + 問資訊	每類一條 regex；先剝問法外圍詞；搜尋 + 資訊內容走純問答	T0.7 194 → 111
4	存活突變體、E2 的 R2 違反、R3 長尾	死碼（lens 的「找來源」被搜尋的「找」先接）；「...好嗎」把任務變成問答；「記事本幫我打開」沒規則收	觸發順序調整；問答判斷先正規化；受詞在前的句型翻回動詞在前（有分隔才直接翻，沒分隔只當最後 fallback）	規則式 81 → 7；模型約 130 → 約 90

第四輪的修法是看 T0.7 那批的清單歸納的：T0.7 降 40%、其他批次降 30%，從一批歸納的修正有七成能推到另一批。對使用者的意思是：「記事本幫我打開」「把截圖整理到資料夾好嗎」這類句子從「花 token 交給模型」或「變成問答不做事」變成本機直接做。

二、測試工具自己也要測

研究過程中測試工具本身被自己的方法抓到四次錯，每次都會改變結論：

1. 測試基座的標籤比對：`computer:key:nexttrack` 被拆成不同結構，「下一首」的 77% 脆弱度有一半是假象。
2. 種子含其他規則的觸發詞：「搜尋台北天氣」改寫後被天氣規則接走，那不是路由的錯。
3. 模型語意漂移：「播晴天」被改成「搜尋晴天」，違反是測試的錯，E1 量化為 3.5%。
4. 分析腳本的去重順序：造成「T0 殺最多、T1 最少」的假結論，差點進報告。

另外，存活突變體的人工分類若沒有「每個判斷附一句可執行的 killer」，第一輪 33% 的誤判也會直接進報告。這些經驗的共同結論：測試生成、測試基座、分析腳本都是程式，都會錯，都要有自己的檢查。

陸、研究限制與效度威脅

- **種子選擇**：41 條原始種子是我挑的，偏向我知道 router 會做的事；46 條 killer 種子是對著存活突變體寫的，偏向 regex 的詞表。兩者都不是真實使用者的句子分佈。
- **單一受測對象**：只有一個 router，結論是否適用於其他 regex 意圖路由未驗證；測試基座已抽象化，可移植。
- **生成句的有效性**：E1 目前只有一位標記者，尚未算 Cohen's κ ；標記者同時是開發者。
- **等價突變體**：只抽 60 個人工估（18%），推回全體有誤差。
- **批次規模**：每個種子 × 關係只生 5 句，溫度的影響可能要更大批次才看得出來；本研究的結論是「在這個規模下看不出」，不是「沒有影響」。
- **我既是開發者也是實驗者**：修 router 時知道測試長什麼樣。緩解：只針對分類後的原因修，不針對個別句子；golden tests 與生成測試分開；第四輪用「看一批、驗其他批」量化推廣性。

柒、結論與未來工作

一、結論

1. **該測什麼**：蛻變關係讓沒有 oracle 的自然語言路由也能自動生成測試；第一次就在 928 句裡找到 433 個手寫測試沒抓到的違反。模型生成器找到的錯是模板的兩倍，改寫有效率 96.5%，模板生成器則是免費、確定、一秒跑完的對照組。
2. **測試多可靠**：溫度在 $n=5$ 的批次規模下對找錯數、錯集合、殺傷力都沒有可見影響，溫度之間的差異與 $T=0$ 重跑的雜訊相同。用模型生成測試的比較必須附重跑底線。
3. **留多少**：以突變體為需求，3,355 條縮到 196 條（5.8%）仍保留 100% 殺傷力，greedy / HGS / irreplaceable 三種策略幾乎一樣；不跑突變測試的便宜準則跟隨機沒有差別。
4. **先測哪些**：additional 排序 APFD 0.985，跑前 10% 就找到全部的錯；total 排序輸給隨機。
5. **回饋**：四輪修正把規則式批次的違反從 706 降到 7、模型批次從約 385 降到約 90，無反彈；生成的測試批次同時成為回歸集，抓到修正過程中新壞的句子。
6. **測試工具自己也要測**：測試基座、種子、生成器、分析腳本各被抓到一次錯，每次都會改變結論。

二、未來工作

- 找第二位標記者完成 E1，算 Cohen's κ 。
- 從真實使用紀錄補第三批種子，檢驗種子偏誤。
- 第二個受測對象：找一個開源的 regex 意圖路由（如 Home Assistant 的 conversation）跑同一套工具。
- 把批次放大（ $n=20$ ）重做 RQ2，確認溫度是否在更大規模下有影響。
- 第四輪後的 router（986 個突變體）重算突變分數，補進 E6 的表。

- 比較 regex 路由與用模型做路由的可靠度與成本。

參考文獻

1. T. Y. Chen, S. C. Cheung, S. M. Yiu. Metamorphic testing: a new approach for generating next test cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology, 1998.
2. R. A. DeMillo, R. J. Lipton, F. G. Sayward. Hints on test data selection: Help for the practicing programmer. IEEE Computer, 11(4), 1978.
3. Y. Jia, M. Harman. An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering, 37(5), 2011.
4. M. J. Harrold, R. Gupta, M. L. Soffa. A methodology for controlling the size of a test suite. ACM Transactions on Software Engineering and Methodology, 2(3), 1993.
5. G. Rothermel, R. H. Untch, C. Chu, M. J. Harrold. Prioritizing test cases for regression testing. IEEE Transactions on Software Engineering, 27(10), 2001.
6. C.-T. Lin, K.-W. Tang, G. M. Kapfhammer. [Test suite reduction methods that decrease regression testing costs by identifying irreplaceable tests](#). Information and Software Technology, 2014.
7. C.-T. Lin, K.-W. Tang, J.-S. Wang, G. M. Kapfhammer. [Reducing the cost of regression testing by identifying irreplaceable test cases](#). 2012.
8. 本研究程式碼與數據：github.com/Venshen885361/jarvis-desktop，research/ 目錄。

※ 第 1-5 條為經典文獻，卷期頁碼請交稿前依學校格式核對；第 6-7 條已核對作者與標題。